

1 · Importance of TD Methods in RL

Why TD is Central to RL

TD methods are the most important class of RL algorithms. They are:

- **Model-free:** learn directly from experience — no $p(s', r|s, a)$ needed.
- **Online:** update at every step — no need to wait for episode end.
- **Bootstrapping:** use the current estimate of the next state's value to update the current state's value.

This combination makes TD practical for large, real-time problems where MC is too slow (must wait for episode) and DP is infeasible (model unavailable).

2 · How TD Connects to DP and MC

The Triangle: DP vs MC vs TD

Property	Dynamic Programming	Monte Carlo	TD
Needs model?	Yes	No	No
Bootstraps?	Yes	No	Yes
Online (step-by-step)?	Yes (sweep)	No (end of episode)	Yes
Target	$\mathbb{E}[R + \gamma V(S')]$	Full return G_t	$R_{t+1} + \gamma V(S_{t+1})$
Bias	Low	None	Some (due to bootstrap)
Variance	Low	High	Low

TD borrows from both: uses experience like MC (model-free) and bootstraps like DP (one-step lookahead). This is the key insight — it is “online MC + bootstrapping”.

The update targets compared:

DP target: $\sum_{s', r} p(s', r|s, a) [r + \gamma V(s')]$ (uses model, exact expectation)

MC target: $G_t = R_{t+1} + \gamma R_{t+2} + \dots$ (actual return, no bootstrap)

TD(0) target: $R_{t+1} + \gamma V(S_{t+1})$ (one sample, one bootstrap step)

3 · Framework of TD(0) — The Basic TD Method

TD(0) Prediction Update

$$V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\delta_t = \text{TD error}}$$

- $\delta_t = \text{TD error}$ — how surprising the transition was (positive: better than expected, negative: worse).
- $R_{t+1} + \gamma V(S_{t+1})$ = the **TD target** (our improved estimate using next state).
- Update happens *every step*, not at episode end.
- Converges to $v_\pi(s)$ as long as α decreases appropriately.

Worked Example — TD(0) update trace

Given: $V(S_t) = 0.5$, $R_{t+1} = 1$, $V(S_{t+1}) = 0.8$, $\alpha = 0.1$, $\gamma = 0.9$.

innovate

achieve

lead

$$\delta_t = 1 + 0.9 \times 0.8 - 0.5 = 1 + 0.72 - 0.5 = 1.22$$

$$V(S_t) \leftarrow 0.5 + 0.1 \times 1.22 = \mathbf{0.622}$$

The state was better than expected (large positive reward + good next state) — value rises.

If $R_{t+1} = -1$, $V(S_{t+1}) = 0.2$:

$$\delta_t = -1 + 0.9 \times 0.2 - 0.5 = -1 + 0.18 - 0.5 = -1.32$$

$$V(S_t) \leftarrow 0.5 + 0.1 \times (-1.32) = \mathbf{0.368}$$

Worse than expected — value falls.

4 · SARSA — On-Policy TD Control

Goal: Learn $q_\pi(s, a)$ (action-value function) to derive a control policy. We need action-values (not state-values) since we have no model to compute $\arg \max_a$.

SARSA Update Rule

After transition $(S_t, A_t) \xrightarrow{R_{t+1}} (S_{t+1}, A_{t+1})$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$$

Name: State, Action, Reward, State, Action — the five elements of the update tuple.

Why on-policy?

The next action A_{t+1} is sampled from the *current policy* π (e.g. ϵ -greedy). The update uses $Q(S_{t+1}, A_{t+1})$ — the value of the action *actually taken next*. So SARSA evaluates and improves the *same* policy that generates experience.

SARSA algorithm (pseudocode):

1. Initialise $Q(s, a)$ arbitrarily; $Q(\text{terminal}, \cdot) = 0$.
2. For each episode: initialise S ; choose A from Q using ϵ -greedy.
3. Repeat each step: take A , observe R, S' ; choose A' from $Q(S', \cdot)$ using ϵ -greedy.
4. Apply SARSA update. Set $S \leftarrow S'$, $A \leftarrow A'$.
5. Until S is terminal.

5 · Q-Learning — Off-Policy TD Control

Q-Learning Update Rule

After transition $(S_t, A_t) \xrightarrow{R_{t+1}} S_{t+1}$:

$$Q(S_t, A_t) \leftarrow Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$$

Key difference from SARSA: uses $\max_a Q(S_{t+1}, a)$, not $Q(S_{t+1}, A_{t+1})$. It always bootstraps from the *best* possible next action, regardless of what the behaviour policy actually does.

Why off-policy?

The target $\max_a Q(S_{t+1}, a)$ is the Bellman optimality equation — it assumes the agent will act greedily from the next state. But the actual next action A_{t+1} may be exploratory. Q-Learning learns q^* *directly*, independently of the policy being followed. The behaviour policy just needs to cover all (s, a) pairs.

6 · SARSA vs. Q-Learning — Key Differences

Side-by-Side Comparison

Property	SARSA	Q-Learning
Update uses	$Q(S_{t+1}, A_{t+1})$ — actual next action	$\max_a Q(S_{t+1}, a)$ — best possible action
Policy type	On-policy	Off-policy
What it learns	q_π (value of <i>followed</i> policy)	q^* (value of <i>optimal</i> policy)
Behaviour on Cliff Walk	Takes safer inland path (cautious)	Falls off cliff under ϵ -greedy (learns optimal but risky path)
Convergence	Converges to q^* under GLIE	Converges to q^* directly
Safe / cautious?	Yes — accounts for actual exploration	No — ignores exploration risk

Cliff Walking Example — SARSA vs Q-Learning (must know)

Setup: Grid world. Bottom row has a cliff. Agent starts at bottom-left, goal is bottom-right. Falling off cliff: reward -100 , restart. All other moves: reward -1 .

Q-Learning learns the *theoretically* optimal path — walk along the cliff edge (fewest steps, reward ≈ -13). But under ϵ -greedy, the agent occasionally steps off the cliff (getting -100). *Online performance is poor.*

SARSA accounts for the exploration policy. It “knows” the agent sometimes makes random moves, so it learns to avoid the cliff edge — takes the safer inland path (reward ≈ -17 but stays safe during learning). *Online performance is better.*

Exam takeaway: SARSA = safe during training (on-policy). Q-Learning = finds optimal but risky path during training. As $\epsilon \rightarrow 0$, both converge to the same optimal policy.

Worked Trace — SARSA vs Q-Learning on same transition

Given: $Q(S_t, A_t) = 2.0$, $\alpha = 0.1$, $\gamma = 0.9$, $R_{t+1} = 1$.

Next state S_{t+1} has $Q(S_{t+1}, \text{left}) = 3.0$, $Q(S_{t+1}, \text{right}) = 5.0$.

Actual next action (by ϵ -greedy): $A_{t+1} = \text{left}$ (exploratory choice).

SARSA (uses actual $A_{t+1} = \text{left}$):

$$Q(S_t, A_t) \leftarrow 2.0 + 0.1[1 + 0.9 \times 3.0 - 2.0] = 2.0 + 0.1[1 + 2.7 - 2.0] = 2.0 + 0.1 \times 1.7 = \mathbf{2.17}$$

Q-Learning (uses $\max = 5.0$):

$$Q(S_t, A_t) \leftarrow 2.0 + 0.1[1 + 0.9 \times 5.0 - 2.0] = 2.0 + 0.1[1 + 4.5 - 2.0] = 2.0 + 0.1 \times 3.5 = \mathbf{2.35}$$

Q-Learning gives a higher estimate (more optimistic) because it uses the *best* possible next action, not the one actually taken.

7 · Maximization Bias and Double Q-Learning

7.1 · The Problem: Maximization Bias

What is Maximization Bias?

Q-Learning uses $\max_a Q(S_{t+1}, a)$ in its update. But $Q(s, a)$ is a *noisy estimate* of $q^*(s, a)$.

The bias:

$$\mathbb{E}[\max_a Q(s, a)] \geq \max_a \mathbb{E}[Q(s, a)] = \max_a q^*(s, a)$$

The expected maximum of noisy estimates is **always** $\geq$ the true maximum.

Why? Even if all actions have the same true value, random noise will make some look better. Selecting the maximum *amplifies* that noise — we systematically pick the overestimated action.

Impact on learning:

- Agent is over-optimistic about values of certain states/actions.
- Can cause the agent to commit to suboptimal actions because they *look* good early on.
- Performance degrades or converges more slowly.
- More severe when there are many actions with similar true values.

Illustrative Example — Maximization Bias

Setup: One state A , two actions: **left** (goes to state B) and **right** (terminal, reward = 0). State B has many actions, each with reward drawn from $\mathcal{N}(-0.1, 1)$ (mean = -0.1 , noisy).

True value: $Q(A, \text{left}) \approx -0.1$ (negative in expectation). $Q(A, \text{right}) = 0$.

What Q-Learning does early: With few samples, some actions in B appear positive (lucky noise). So $\max_a Q(B, a) > 0$, making $Q(A, \text{left}) > 0$. Agent incorrectly prefers **left** over **right**.

Maximization bias: Q-Learning overestimates $Q(A, \text{left})$ due to noise.

7.2 · Solution: Double Q-Learning

Double Q-Learning — Separating Selection and Evaluation

Maintain two independent value estimates Q_1 and Q_2 .

At each step, with probability 0.5:

$$Q_1(S_t, A_t) \leftarrow Q_1(S_t, A_t) + \alpha [R_{t+1} + \gamma Q_2(S_{t+1}, \arg \max_a Q_1(S_{t+1}, a)) - Q_1(S_t, A_t)]$$

Otherwise swap $Q_1 \leftrightarrow Q_2$ in the update.

Key idea: Q_1 selects the action ($\arg \max_a Q_1$); Q_2 evaluates it. Since Q_1 and Q_2 are independent, the bias is removed:

$$\mathbb{E}[Q_2(s, \arg \max_a Q_1(s, a))] = q^*(s, \arg \max_a q^*(s, a))$$

No systematic overestimation.

8 · Race Car Example with TD

Race Track with SARSA and Q-Learning

State: (x, y, v_x, v_y) — position + velocity. **Actions:** 9 acceleration combinations. **Reward:** -1 per step; going off-track: -1 + return to start.

TD vs. MC on the race track:

	MC (from Session 8)	TD (this session)	
When it updates	End of each lap	Every step (online)	achieve
Episodes needed	Many full laps required	Learns even from partial runs	
Handling crashes	Episode ends; returns -1 per step	Updates immediately on crash re-	lead
		ward	
Practical speed	Slow for long tracks	Much faster convergence	

SARSA on race track:

Agent follows ϵ -greedy policy. After each step, updates $Q(s, a)$ using actual next action. SARSA learns to *decelerate before corners* — it accounts for exploration errors (sometimes veering off-track) and prefers safer trajectories.

Q-Learning on race track:

Uses $\max_a Q(S_{t+1}, a)$ at each step. Learns the theoretically optimal racing line (fastest path). However, during learning under ϵ -greedy, it may crash more often on corners because it ignores the exploration risk — analogous to the Cliff Walking result.

Simplified 1-D Race Car Trace — Q-Learning

States: positions 0–6, velocity $v \in \{1, 2\}$. **Actions:** Acc or Hold. **Reward:** $-1/\text{step}$. $\alpha = 0.1$, $\gamma = 0.9$. Initialise all $Q(s, a) = 0$.

Step 1: In state (0, 1), take Acc. Move to (2, 2). Reward = -1 .
 $\max_a Q((2, 2), a) = 0$.

$$Q((0, 1), \text{Acc}) \leftarrow 0 + 0.1[-1 + 0.9 \times 0 - 0] = -0.1$$

Step 2: In state (2, 2), take Hold. Move to (4, 2). Reward = -1 .
 $\max_a Q((4, 2), a) = 0$.

$$Q((2, 2), \text{Hold}) \leftarrow 0 + 0.1[-1 + 0.9 \times 0 - 0] = -0.1$$

Step 3: In state (4, 2), take Hold. Reach finish (6, 2). Reward = -1 . Terminal: $\max Q(\text{finish}) = 0$.

$$Q((4, 2), \text{Hold}) \leftarrow 0 + 0.1[-1 + 0 - 0] = -0.1$$

S1-2026-27 Work Integrated Learning Programmes S1-2026-27

After this first episode: values = -0.1 along the path. After many episodes, values become increasingly negative for states far from finish (more steps $\Rightarrow$ lower return). Greedy policy: Acc whenever possible (fewer steps $\Rightarrow$ higher value $\approx$ $-$ fewer steps).

9 · Exam Key-Takeaways — Sessions 9 & 10

Must Know — TD Methods

1. **TD = model-free + bootstrapping + online.** Combines the best of MC and DP.
2. **TD(0) update:** $V(S_t) \leftarrow V(S_t) + \alpha \underbrace{[R_{t+1} + \gamma V(S_{t+1}) - V(S_t)]}_{\text{TD target}}$.
3. **TD error** $\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$ — know this symbol.
4. **SARSA** (on-policy): update uses actual next action A_{t+1} . $Q(S, A) \leftarrow Q(S, A) + \alpha [R + \gamma Q(S', A') - Q(S, A)]$.
5. **Q-Learning** (off-policy): update uses $\max_a Q(S', a)$, not actual A' . Learns q^* directly regardless of behaviour policy.
6. **Cliff Walk:** SARSA = safe inland path; Q-Learning = risky optimal path. SARSA has better *online* performance under ε -greedy.
7. **Maximization bias:** $\mathbb{E}[\max_a Q(s, a)] \geq \max_a q^*(s, a)$. Q-Learning is over-optimistic; hurts convergence and safety.
8. **Double Q-Learning:** two independent estimators — one selects action, other evaluates it. Removes bias.
9. **Race track with SARSA:** conservative (decelerate near corners); Q-Learning: finds fastest line but more crashes during training.
10. **Key difference SARSA vs Q-Learning** in one word: SARSA is *cautious*; Q-Learning is *optimistic*.

DRL EC3 Exam Handout — Sessions 9 & 10 (Temporal Difference Learning). Ref: Sutton & Barto Ch. 6.



S1-2026-27 Work Integrated Learning Programmes S1-2026-27